

A more complex state, debugging React apps

Complex State

- In our previous example, the application state was simple as it was comprised of a single integer. What if our application requires a more complex state?
- Default and basic declaration of multiple state;

```
const [left, setLeft] = useState(0);  
const [right, setRight] = useState(0);
```

- Improvised declaration

```
const [clicks, setClicks] = useState({ left: 0, right: 0 })
```

- stateHandlers

```
const handleLeftClick = () => {  
  const newClicks = {  
    left: clicks.left + 1,  
    right: clicks.right  
  }  
  setClicks(newClicks)  
};
```

- Improvised stateHandlers

```
const handleLeftClick = () => setClicks({ ...clicks, left: clicks.left + 1 });  
vise versa for right
```

- Some readers might be wondering why we didn't just update the state directly, like this:?

```
const handleLeftClick = () => {  
  clicks.left++  
  setClicks(clicks)  
}
```

- Solution

- it is forbidden in React to mutate state directly, since it can result in unexpected side effects.

- Changing state has to always be done by setting the state to a new object.
- If properties from the previous state object are not changed, they need to simply be copied
- which is done by copying those properties into a new object and setting that as the new state.
-

- **Imp Note**

Storing all of the state in a single state object is a bad choice for this particular application;

Handling arrays

- Every click is stored in a separate piece of state called allClicks that is initialized as an empty array:

```
const [allClicks, setAll] = useState([]);
```

- Left - When the left button is clicked, we add the letter L to the allClicks array:

```
setAll(allClicks.concat('L'));
```

- Right - When the Right button is clicked, we add the letter R to the allClicks array:

```
setAll(allClicks.concat('R'));
```

Concat method is Immutable, it returns a brand New Array[] with the updated value; the **join method** on the allClicks array, that joins all the items into a single string, separated by the string passed as the function parameter, which in our case is an empty space.

- This is also one possibility but ,Why is this code fundamentally broken? (What rule does it break?)

```
const handleLeftClick = () => {  
  allClicks.push('L')  
  setAll(allClicks)  
  setLeft(left + 1)  
}
```

the state of React components, like allClicks, must not be mutated directly. Even if mutating state appears to work in some cases, it can lead to problems that are very hard to debug.

- React looks at the box (the memory reference), not what is inside the box.
- If you use .push(), it is the exact same box, so React ignores it.
- By using .concat(), you hand React a brand new box, and it instantly redraws the screen.
- Bonus Pro-Tip: The course teaches .concat(), which is perfectly correct. But in modern React jobs, you will actually see developers use the exact same Spread Operator trick you used for Objects:

```
setClicks([...clicks, 'L']);
```

- This means: "Create a new array, dump everything from the old clicks array into it, and then add 'L' at the end." Both methods do the exact same thing (create a new array), so use whichever one makes more sense to your brain!

Update of the state is asynchronous

- Course asked to create total state of left right clicked okay

```
const [clicks, setClicks] = useState({ left: 0, right: 0 })
const [total, setTotal] = useState(0);
> setTotal(clicks.left + clicks.right)
```

- Output

Left : 4 Right : 3

L L R L R R L

total : 6

- something wrong in output total should print 7 but it is printing 6;

Lets find out why in the upcoming sections Rendering clicks by value L:0 | R:0

left before 0 left after 0

Rendering clicks by value L:1 | R:0

- here clicked happend but value didnt changed in the actual clicks.left object
- but the in handleLeftClicked function its chnaging instantly

it should print after very first clicked: left before 0 left after 1

total should print 1 but it didnt.

but didnt chnage let see why ??

- The reason for this is that a state update in React happens **asynchronously**.
- not immediately but "at some point" after the current component function is finished, before the component is rendered again.
- By Mentor :-

```
setClicks({ ...clicks, left: clicks.left + 1 });
console.log('left after', clicks.left); // Prints the OLD number!
```

```
setTotal(clicks.left + clicks.right); // Uses the OLD number!
```

- In standard JavaScript, if you change a variable, it updates instantly on the very next line. React state does NOT work like that.
- When you run `setClicks(...)`, you are not instantly changing the `clicks` variable. You are just sending a request to the React Engine saying, "Hey, next time you redraw the screen, make the `clicks` variable equal this new number."
- But while the rest of your `handleLeftClick` function finishes running, the `clicks` variable is locked in time.
- It still holds the old data. That is why your `console.log` prints the old number, and your `setTotal` calculates using the old number!
- Because you declared it as a constant (`const [clicks, setClicks] = ...`), the value of `clicks` is completely locked in **scope for that specific run of the function**.

Solution

- You cannot rely on the state variable immediately after you "set" it
- Instead, you calculate the new number in a normal variable first, and then hand that normal variable to React.
- Final Verdict by Mentor on **Update of the state is asynchronous**:-
- So yes, you cannot rely on the **useState variable immediately after updating it**.
- You must "variablize" the update first, just like you did.

Conditional rendering

- It is just if/else logic!
- The `History` component renders completely different React elements depending on the state of the application. This is called conditional rendering.

Old React

- nothing special to note down just general reading material

Debugging React applications

- Good practices and tools for debugging are extremely important for this reason.
- Lucky for us, React is an extremely developer-friendly library when it comes to debugging.
- Before we move on, let us remind ourselves of one of the most important rules of web development.
 - The first rule of web development
 - **Keep the browser's developer console open at all times.**

- The Console tab in particular should always be open, unless there is a specific reason to view another tab. //
 - Keep both your code and the web page open together at the same time, all the time. //
 - we must transform our function into the less compact form and receive the entire props object without destructuring it immediately:
 1. Console.log is your Best Friend
 2. The Comma Rule (Never use +)
 - When you want to log an object, never do this: `console.log('The state is ' + clicks)`
 - Always use a comma: `console.log('The state is ', clicks)`
 3. The debugger; Command
 4. React Developer Tools
-

Rules of Hooks

1. The `useState` function must not be called from inside of a loop, a conditional expression, or any place that is not a function defining a component.
2. To recap, hooks may only be called from the inside of a function body that defines a React component:

```
const App = () => {
  // these are ok
  const [age, setAge] = useState(0)
  const [name, setName] = useState('Juha Tauriainen')

  if ( age > 10 ) {
    // this does not work!
    const [foobar, setFoobar] = useState(null)
  }

  for ( let i = 0; i < age; i++ ) {
    // also this is not good
    const [rightWay, setRightWay] = useState(false)
  }

  const notGood = () => {
    // and this is also illegal
    const [x, setX] = useState(-1000)
  }

  return (
    //...
  )
}
```

Event Handling Revisited

- Why is this illegal? Because you put parentheses () at the end. When React renders the HTML, it sees those parentheses and says, "Oh, you want me to execute this function right now!"
- It will run the function immediately when the page loads, **without waiting for the click!** If that function contains a set... state update, it will cause an infinite loop and crash your app.

1. THE GOOD WAY #1 (Pass by Reference)

```
<button onClick={handleLeftClick}>Click Me</button>
```

2. THE GOOD WAY #2: -> (Inline Arrow Function)

```
// CORRECT
<button onClick={() => console.log('clicked')}>Click Me</button>

// CORRECT
<button onClick={() => setClicks(clicks + 1)}>Click Me</button>
```

- This is also perfectly legal! You are not calling a function here; you are defining a brand new, nameless function right inside the HTML. React says, "Okay, I will save this nameless arrow function for later, and run it when the button is clicked."

A function that returns a function

Closures (Functions returning functions)

- It is a JavaScript concept called Currying or Closures.
- Another way to define an event handler is to use a function that returns a function.

```
const outer = () => {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  };
}

const secrete = outer();
secrete();
```

- inner() function "remembers" the count variable from the outer scope, even after the outer function has finished running.
- React uses that exact same superpower to build customized event handlers.

- Instead of writing three different functions (setToOne, setToTwo, setToThree), you write ONE outer function that takes a parameter, and it returns a closure (an inner function) that remembers that parameter

```
const hello = () => {  
  const handler = () => console.log('hello world'); // This is a function  
  return handler; // It RETURNS the function!  
}
```

- When React sees

, it runs the hello() function immediately on render. But what does it get back? It gets the handler function back!

- So, after React executes hello(), the HTML basically transforms into this: `button`
- Because it successfully handed a function to the onClick, React is perfectly happy, and it will not cause an infinite loop.
- Syntaxes

1. Vanilla JS Syntax (Yours - Perfectly Valid)

```
const hello = () => {  
  return function handler() {  
    setMsg("Hello React")  
  }  
};
```

2. Standard React Arrow Syntax

```
const hello = () => {  
  const handler = () => {  
    setMsg("Hello React")  
  }  
  return handler;  
};
```

3. Ultra-Short Expert Syntax (Hard to read, but you will see it often)

```
const hello = () => () => setMsg("Hello React");
```

Passing Event Handlers to Child Components

```
const Button = ({ onClick, text }) => {
  return <button onClick={onClick}>{text}</button>
}

const App = () => {

  const [msg, setMsg] = useState("");

  const hello = () => () => setMsg("Hello React User , Welcome To react World ,
We are testing now A function that actually returns a function which basic Concept
in JavaScript Called 'Closures'");

  return (
    <div>
      // And used it like this:
      <Display textToShow={msg} />
      <Button onClick={hello()} text="Show Text" />
    </div>
  )
};
```

Do Not Define Components Within Components

```
// This is the right place to define a component
const Button = (props) => (
  <button onClick={props.onClick}>
    {props.text}
  </button>
)

const App = () => {
  const [value, setValue] = useState(10)

  const setToValue = newValue => {
    console.log('value now', newValue)
    setValue(newValue)
  }

  // Do not define components inside another component

  const Display = props => <div>{props.value}</div>

  return (
    <div>

      <Display value={value} />
      <Button onClick={() => setToValue(1000)} text="thousand" />
      <Button onClick={() => setToValue(0)} text="reset" />
    </div>
  )
};
```



```
    <Button onClick={() => setToValue(value + 1)} text="increment" />
  </div>
)
}
```

- Never define components inside of other components
 - The method provides no benefits and only leads to problems.
 - One such problem is that React will treat a component defined inside of another component as a "new component" in every render.
 - This makes it impossible for React to optimize the component.
-

Useful Reading

- The internet is full of React-related material. However, we use the new style of React for which a large majority of the material found online is outdated.
- The official React documentation is worth checking out at some point, although most of it will become relevant only later on in the course.
- - Also, everything related to class-based components is irrelevant to us;